

**CSA0590-DATABASE MANAGEMENT
SYSTEM FOR HYBRID WORKLOADS
ASSIGNMENT-04**

NAME:S.GAYATHERI

REG NO:192521306

Explain pipelining and materialization strategies used in query execution. compare their efficiency.

Pipelining and Materialization Strategies in Query Execution

In a DBMS, query execution involves multiple operations such as selection, projection, join, sorting, grouping, etc.

When a query has multiple operations, the DBMS must decide how to pass intermediate results between operators.

Two important strategies are:

1. Materialization
2. Pipelining

Materialization Strategy

Definition

Materialization is a query execution strategy in which the result of each operation is stored (materialized) in temporary storage before being used by the next operation.

Each operator:

- Executes completely
- Produces a full intermediate result
- Stores it in memory or disk
- Then the next operator reads it

Working

Suppose the query is:

```
SELECT name  
FROM Student  
WHERE marks > 70;
```

Execution steps:

1. Selection (marks > 70)
2. Projection (name)

Using materialization:

- Step 1: Store selected tuples in temporary relation R1.
- Step 2: Read R1 and apply projection.

Example with Join

```
SELECT *  
FROM A JOIN B ON A.id = B.id  
WHERE A.age > 20;
```

Materialization:

1. Perform join → store result in temp table T1.
2. Apply selection on T1.
3. Output final result.

Advantages

- Easy to implement
- Simple debugging
- Can reuse intermediate results
- Useful when result is needed multiple times

Disadvantages

- Requires large temporary storage
- High disk I/O cost
- Slower for large datasets
- Extra time for writing and reading intermediate files

Pipelining Strategy

Definition

Pipelining is a strategy where results are passed directly from one operator to another without storing them completely.

The next operation starts processing as soon as it receives tuples.

Working

Using same query:

```
SELECT name  
FROM Student  
WHERE marks > 70;
```

Pipelining:

- Selection produces one tuple
- Immediately passed to projection
- No temporary table created

Processing happens simultaneously.

Types of Pipelining

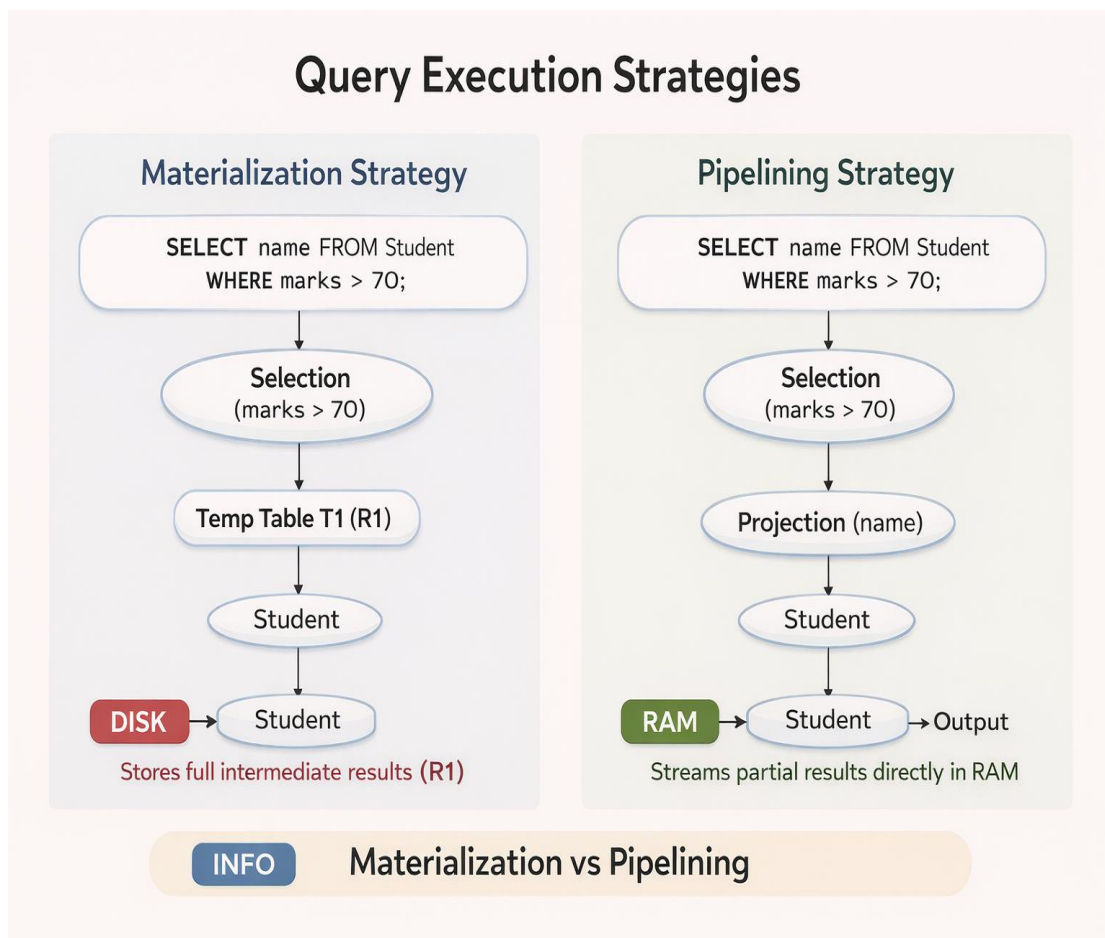
1. Demand-driven (Pull-based)
 - Parent operator requests tuple from child.
2. Producer-driven (Push-based)
 - Child operator sends tuples to parent automatically.

Example with Join

```
SELECT *  
FROM A JOIN B ON A.id = B.id  
WHERE A.age > 20;
```

Pipelining:

1. Join produces one tuple.
2. Immediately passed to selection.
3. If condition satisfied → output.
4. No full join result stored.



Advantages

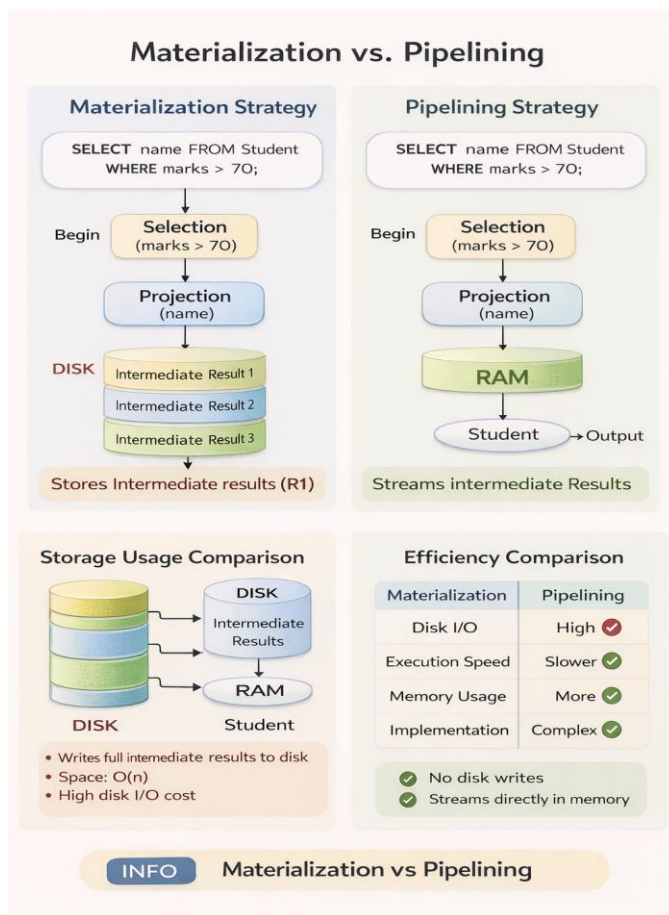
- No temporary storage required
- Less disk I/O
- Faster execution
- Memory efficient
- Suitable for large queries

Disadvantages

- Harder to implement
- Not suitable for all operations (e.g., sorting, grouping)
- Cannot reuse intermediate results easily

Comparison of Materialization and Pipelining

Feature	Materialization	Pipelining
Storage	Requires temporary storage	No temporary storage
Disk I/O	High	Low
Execution Speed	Slower	Faster
Memory Usage	More	Less
Implementation	Simple	Complex
Suitable For	Reusable intermediate results	Real-time processing
Blocking Operations	Works for all	Not good for sorting/group by



Efficiency Comparison

- **Time Complexity**

Both strategies perform same logical operations.

But:

- Materialization → Extra time for writing and reading temp files.
- Pipelining → Avoids extra I/O → Faster.

- **Space Complexity**

- Materialization → $O(n)$ extra space for intermediate results.
- Pipelining → Minimal extra space.

- **Disk Cost**

- Materialization → High disk access.
- Pipelining → Reduces disk cost significantly.

When to Use Which?

Use Materialization When:

- Intermediate result reused multiple times
- Debugging required
- Operation is blocking (e.g., sorting, aggregation)

Use Pipelining When:

- Large datasets
- Performance is important
- Operations are non-blocking (selection, projection, join)

Materialization and pipelining are two important query execution strategies in DBMS.

- Materialization stores intermediate results and is simple but costly in terms of time and storage.
- Pipelining passes results directly between operators, reducing disk I/O and improving performance.

In modern database systems, pipelining is preferred for performance, but materialization is still used for blocking operations like sorting and grouping.

Thus, choosing the right strategy improves query efficiency and overall system performance.